Agentic Observability & Self-Healing Infrastructure

Production-Grade Reliability Layer for AGI Systems | Meta Global AI Developer Hackathon 2026 - Indie Track

Author: Digital Business Architect & AIOps Specialist (BO01OK) | Stack: Meta Llama Models, OpenTelemetry, Python



1. Executive Summary

This system solves the #1 reason agent systems fail in production: lack of deterministic observability and self-healing. While most hackathon demos focus on agent intelligence, we focus on agent reliability. Our thesis: AGI won't fail due to lack of intelligence, but due to lack of infrastructure. We build that infrastructure using Meta's Llama models as the orchestrator with full OpenTelemetry tracing and an autonomous remediation engine.

2. Problem Statement - Why Agents Fail

- Infinite Loops: No step limits or state validation
- Black-Box Traces: No visibility into thought -> tool -> handoff
- No Failure Handling: One tool timeout collapses entire swarm
- Cost Explosion: No token/latency guardrails
- Non-Deterministic Outputs: No schema validation

3. Solution Architecture - 10 Components

Layer 1: Orchestration (Llama)
- Llama 4 via Meta Model API for reliable tool-calling and long-context trace analysis
- Pydantic schemas for deterministic I/O

Layer 2: Guardrail Layer (Critical for Judges)
- Max 15 steps/agent, 30s/tool timeout, circuit breaker after 3 failures, JSON schema validation

Layer 3: Worker Agents Swarm
- Specialized agents (researcher, coder, analyst) orchestrated via LangGraph

Layer 4-8: Observability & Self-Healing Pipeline
- OpenTelemetry Collector: captures every thought, tool_call, latency, token
- Observability Store: Postgres + Vector DB for trace search
- Anomaly Detection: heuristic + LLM-based detection (latency spike, error rate >20%, cost anomaly)
- Self-Healing Engine: 4 remediation patterns
- Remediation Actions: retry with backoff, fallback model, re-plan, human escalation with trace bundle

Layer 9-10: Interface & Dashboard
- Real-time dashboard: success rate, avg latency, cost per run, failure breakdown

4. Data Flow (Deterministic)

1. User Request -> 2. Orchestrator validates with Pydantic -> 3. Guardrail checks limits -> 4. Worker agents execute (traced) -> 5. OTel Collector logs -> 6. Anomaly Detection scans -> 7. If anomaly -> Self-Healing Engine -> 8. Remediation -> 9. Retry or Escalate -> 10. Response with full trace + health metrics

5. Self-Healing Patterns (Production-Grade)

Pattern 1: Exponential Backoff Retry - For transient tool failures (2s, 4s, 8s)
Pattern 2: Model Fallback - Llama 405B -> Llama 70B -> Llama 8B for cost/latency
Pattern 3: Re-Planning - If agent stuck, Llama re-plans with error context injected
Pattern 4: Human Escalation - After 3 failed remediations, bundle traces + error + suggestion and push to Slack/Email

6. Observability Metrics (What Judges See)

- Trace Completeness: 100% of steps traced
- MTTR (Mean Time To Remediate): < 15s
- Cost per Successful Run: tracked per $/1k tokens
- Failure Mode Classification: timeout, validation, tool_error, infinite_loop
- Guardrail Effectiveness: % runs saved by guardrail vs collapsed

7. Tech Stack & Why Llama

Meta Llama Models: Best open model for tool-calling + long context for trace analysis. Using $150 API credits for hackathon. OpenTelemetry: Industry standard for production observability. Pydantic + Python: Deterministic boundaries. LangGraph: Explicit state machine for agents (not black-box framework). Docker + FastAPI: Production deployable from Day 1.

8. Security & Production Readiness

- No secrets in code, env vars only
- Rate limiting on Meta API
- Input sanitization + output validation
- Circuit breaker prevents cost explosion
- All traces PII-redacted before storage

9. 10-Day Build Plan (Current: Day 1 Done)

Day 1: Skeleton + Tracing (DONE) - main.py + OTel collector
Day 2-3: Guardrail Layer + Failure Injection Tests
Day 4-5: Llama Integration + Self-Healing Engine Patterns 1-2
Day 6: Anomaly Detection + Dashboard
Day 7-8: Patterns 3-4 + Demo Video
Day 9: Polish, Load Test (100 concurrent agents), README
Day 10: Final Submission + Write-up

10. Business Value - Why This Wins

For Enterprise: Turns fragile agent demos into production systems. For AGI Vision: This is the infrastructure layer AGI needs - reliable, observable, self-healing. Judges Reward: Architectural judgment + deterministic boundaries + failure-mode handling + production signal (exact criteria from Meta 2026 rubric). Not just 'building', but building reliably.

11. Links & Contact

GitHub: github.com/BO01OK/agentic-observability-llama
Reddit: r/machinelearningnews post (5S810v6eqC)
Positioning: Digital Business Architect & AIOps Specialist building reliable, observable, and increasingly autonomous AI systems.
Open to 100% Remote Roles: Digital Business Architect, AIOps Architect, AI Systems / Agent Infrastructure